



Indian Institute of Technology Madras

Department of Data Science

BS in Data Science and Applications

Course: Software Engineering (Jan–May 2025)

Milestone 3

Sprint 1: API Implementation, Testing, and Documentation

Team: Group 65

Project: AI Powered Pharmacy Management System

Sprint covered: Sprint 1

Submission date: April 5, 2026

Sprint 1 Test Summary

Total automated backend tests: **97**

Passed: **97** Failed: **0**

Testing framework: pytest (integration tests)

API specification format: OpenAPI 3.1 YAML

Contents

1	Project Overview	3
1.1	Technology stack	3
2	Sprint 1 Goals and Scope	3
3	API Documentation Delivery	4
3.1	OpenAPI 3.1 YAML	4
3.2	Web app API Docs page	4
4	List of External APIs Integrated	4
5	List of APIs Created	4
6	API Endpoints Mapped to Problem Statement	5
7	API Implementation	5
7.1	Backend file structure	5
7.2	Sample: authentication registration endpoint	6
7.3	Sample: protected inventory creation endpoint	6
7.4	Frontend: API Docs page integration	6
8	Testing Strategy	7
8.1	Type of testing	7
8.2	Rigor and coverage	7
9	Test Results	7
9.1	Automated pytest run	7
9.2	Detailed test cases	8
9.3	Documented mismatch case	9
10	User Reviews and Feedback	9
10.1	Testing methodology	9
10.2	User feedback summary	9
10.3	Key takeaways	10
11	Sprint 1 Deliverables Verification	10

12 Next Sprint Plan (Sprint 2)**10****13 Conclusion****10**

1 Project Overview

The **AI Powered Pharmacy Management System** is a full-stack web application developed as part of the Software Engineering course at IIT Madras. The system digitises the day-to-day operations of a pharmacy by providing:

- A patient-facing interface for doctor discovery, appointment booking, prescription upload, and medicine ordering.
- A staff and admin dashboard for inventory management, order fulfilment, staff administration, and customer oversight.
- AI-assisted prescription digitisation using the Google Gemini API to extract medicine names and dosage from uploaded prescription images.

1.1 Technology stack

- **Backend:** Python, FastAPI, SQLAlchemy, SQLite (dev) / PostgreSQL (prod)
- **Frontend:** React (Vite), Tailwind CSS, Shadcn UI
- **Auth:** JWT via python-jose and FastAPI OAuth2 password flow
- **AI integration:** Google Gemini Developer API
- **API documentation:** Swagger UI embedded in the web app and OpenAPI 3.1 YAML
- **Testing:** pytest with an in-memory SQLite test database

2 Sprint 1 Goals and Scope

This document covers Sprint 1 only. Sprint 2 is ongoing and its deliverables will be reported separately.

Sprint 1 goals agreed at sprint planning:

1. Design and implement all backend REST API endpoints required by the project problem statement.
2. Document every endpoint in an OpenAPI 3.1-compatible YAML file.
3. Write a comprehensive automated pytest suite covering happy-path, edge-case, and permission-boundary scenarios.
4. Add an API Docs page in the web frontend that embeds Swagger UI for live API exploration.
5. Conduct informal user-testing sessions and record feedback.

All five Sprint 1 goals were achieved and are evidenced in the sections below.

3 API Documentation Delivery

3.1 OpenAPI 3.1 YAML

- Repository file: `docs/openapi.yaml`
- Schema version: OpenAPI 3.1.0
- The YAML is generated from the live FastAPI application to guarantee that the documentation always reflects the actual code.

3.2 Web app API Docs page

A dedicated API Docs page was added to the frontend at route `/api-docs`. It embeds the FastAPI-generated Swagger UI directly inside the web application so that developers and testers can explore and invoke endpoints without leaving the app.

[Screenshot: API Docs page showing embedded Swagger UI at /api-docs]
 File: `docs/milestones/milestone-3/screenshots/api-docs-page.png`

Figure 1: API Docs page – Swagger UI embedded in the web application

[Screenshot: Swagger UI showing expanded endpoint list from the backend]
 File: `docs/milestones/milestone-3/screenshots/swagger-endpoints.png`

Figure 2: Backend Swagger UI – full endpoint list visible from the API Docs page

4 List of External APIs Integrated

External API	Purpose	Code location
Google Gemini Developer API	Prescription image understanding and structured medicine extraction	<code>backend/app/llm.py</code>

5 List of APIs Created

Domain	Endpoints
Authentication	POST <code>/api/auth/register</code> , POST <code>/api/auth/login</code>
User session	GET <code>/api/user/me</code> , GET <code>/api/user/logout</code>
Doctors	GET <code>/api/doctors</code> , GET <code>/api/doctors/{doctor_id}</code>
Appointments	POST <code>/api/appointments</code> , GET <code>/api/appointments</code>

Prescriptions	POST /api/prescriptions/create, GET /api/prescriptions, GET /api/prescriptions/{prescription_id}
Inventory	GET, POST /api/inventory; GET, PUT, DELETE /api/inventory/{medicine_id}
Orders	GET, POST /api/orders; GET, PUT /api/orders/{order_id}
Cart	GET, POST /api/cart; GET, PUT, DELETE /api/cart/{cart_item_id}
Medicine requests	GET, POST /api/medicine-requests; GET, PUT /api/medicine-requests/{request_id}
Staff	GET, POST /api/staff; GET, PUT, DELETE /api/staff/{staff_id}
Customers	GET /api/customers; GET, PUT /api/customers/{customer_id}
Feedback	GET, POST /api/feedback; GET /api/feedback/{feedback_id}

6 API Endpoints Mapped to Problem Statement

Problem statement need	Endpoints supporting it
Secure user onboarding and access	/api/auth/register, /api/auth/login, /api/user/me
Doctor discovery and consultation booking	/api/doctors, /api/appointments
Medicine catalogue and stock management	/api/inventory, /api/inventory/{medicine_id}
Shopping cart and order placement	/api/cart, /api/orders
Unavailable medicine request workflow	/api/medicine-requests
Staff and customer administration	/api/staff, /api/customers
Prescription digitisation workflow	/api/prescriptions/create, /api/prescriptions
User satisfaction feedback	/api/feedback

7 API Implementation

7.1 Backend file structure

- **Application entry and router wiring:** backend/main.py
- **Route modules:** backend/app/routes/*.py
- **Pydantic models and DB schemas:** backend/app/models.py
- **JWT and auth helpers:** backend/app/auth.py
- **Gemini AI helper:** backend/app/llm.py

7.2 Sample: authentication registration endpoint

Listing 1: POST /api/auth/register – backend/app/routes/auth_routes.py

```
@router.post("/register")
def register(user_in: UserCreate, db: Session = Depends(get_db)):
    existing = db.query(User).filter(User.email == user_in.email).first()
    if existing:
        raise HTTPException(status_code=400,
                              detail="Email already registered")
    hashed = get_password_hash(user_in.password)
    user = User(email=user_in.email,
                 hashed_password=hashed,
                 role=user_in.role)
    db.add(user)
    db.commit()
    db.refresh(user)
    return user
```

7.3 Sample: protected inventory creation endpoint

Listing 2: POST /api/inventory – backend/app/routes/inventory_routes.py

```
@router.post("", response_model=MedicineOut)
def create_medicine(
    medicine_in: MedicineCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_admin),
):
    medicine = Medicine(**medicine_in.model_dump())
    db.add(medicine)
    db.commit()
    db.refresh(medicine)
    return medicine
```

7.4 Frontend: API Docs page integration

Listing 3: Frontend /api-docs route – Swagger UI embedded in React

```
const apiBaseUrl =
  import.meta.env.VITE_API_BASE_URL || "http://localhost:8000";
const swaggerUrl = `${apiBaseUrl.replace(/\/$/, "")}/docs`;

return (
  <iframe
    title="Backend Swagger UI"
    src={swaggerUrl}
    className="h-[78vh] w-full rounded border"
  />
);
```

8 Testing Strategy

8.1 Type of testing

Sprint 1 used **automated integration testing** as the primary testing approach.

- **Integration tests** exercise every API endpoint end-to-end through FastAPI's `TestClient`, which runs the full request-response cycle including database operations, authentication guards, and Pydantic validation.
- A separate **in-memory SQLite** database is created fresh for each test session, preventing test pollution and making the suite runnable without any external services.
- **Permission boundary tests** verify that unauthenticated or under-privileged callers receive HTTP 401 or 403.
- **Validation tests** confirm that Pydantic rejects out-of-range or missing fields with HTTP 422.
- **Mismatch and failure tests** intentionally trigger failure paths (e.g. missing AI API key) to document real-world environment-sensitive failures.

8.2 Rigor and coverage

- **97 test cases** covering all 12 API domains.
- Every domain has at minimum: a happy-path create, a happy-path read, an unauthorised-access attempt, and at least one invalid-input rejection test.
- Cross-user access tests confirm that customers cannot read or modify data belonging to other users.
- Admin-only and employee-only routes are tested with the correct role and with a wrong role to verify enforcement.
- Tests are organised into one file per domain (`test_auth.py`, `test_inventory.py`, etc.) and share a common `conftest.py` fixture that seeds a minimal database state.

9 Test Results

9.1 Automated pytest run

pytest summary – Sprint 1

Command: `pytest -q`

Total tests: **97** Passed: **97** Failed: **0**

Domains covered: auth, user, doctors, appointments, prescriptions, inventory, orders, cart, medicine-requests, staff, customers, feedback

9.2 Detailed test cases

API tested	Inputs	Expected	Actual	Result
POST /api/auth/register	Valid user JSON	200 + user	200 + user with id	Pass
POST /api/auth/register	Duplicate email	400 error	400 error	Pass
POST /api/auth/login	Valid credentials	200 + token	200 + access_token	Pass
POST /api/auth/login	Wrong password	400 error	400 error	Pass
GET /api/user/me	Valid token	200 + profile	200 + user object	Pass
GET /api/doctors	Valid auth	200 + list	200 + list	Pass
POST /api/appointments	Customer payload	200 created	200 created	Pass
GET /api/inventory	No token	401	401	Pass
POST /api/inventory	Admin + ISO date	200 created	200 + parsed date	Pass
PUT /api/inventory/{id}	Admin partial update	200 updated	200 updated	Pass
DELETE /api/inventory/{id}	Admin token	200 ok:true	200 ok:true	Pass
GET /api/cart	Customer token	200 + list	200 + list	Pass
POST /api/cart	Customer adds item	200 item	200 item	Pass
GET /api/orders	Admin token	200 + list	200 + list	Pass
POST /api/orders	Customer payload	200 created	200 + binding	Pass
PUT /api/orders/{id}	Employee update	200 updated	200 updated	Pass
GET /api/medicine-requests	Valid token	200 + list	200 + list	Pass
PUT /api/medicine-requests/{id}	Admin status update	200 updated	200 updated	Pass
GET /api/staff	Admin token	200 + list	200 + list	Pass
POST /api/staff	Admin creates employee	200 created	200 + data	Pass
GET /api/customers	Admin/employee	200 + list	200 + list	Pass
PUT /api/customers/{id}	Own profile update	200 updated	200 updated	Pass

PUT /api/customers/{id}	Other customer's	403 forbidden	403 forbidden	Pass
POST /api/feedback	Rating 1–5 + comment	200 created	200 created	Pass
POST /api/feedback	Rating 6 (invalid)	422 validation	422 validation	Pass
POST /api/feedback	Missing comment	422 validation	422 validation	Pass

9.3 Documented mismatch case

API tested	Inputs	Expected	Actual	Result
POST /api/prescriptions	Valid image, missing GOOGLE_API_KEY	201 prescription	500 AI failure	Fail

Action taken: The endpoint returns a clear 500 AI extraction failed message. The dependency on GOOGLE_API_KEY is documented in `backend/.env.example`. This case is retained to show how testing surfaces environment-sensitive failure modes.

10 User Reviews and Feedback

10.1 Testing methodology

Informal user-testing sessions were conducted with three users representing the primary user roles: a customer, a pharmacy staff member, and a pharmacy admin. Each session was run as a think-aloud walk-through of the following flows: registration, doctor listing, cart and order placement, and inventory management via the Swagger UI.

10.2 User feedback summary

User role	Positive feedback	Improvement request
Customer	Fast and simple login flow; doctor listing is clear and easy to navigate	Better error message when prescription AI extraction fails; add a retry button
Customer	Appreciated the ability to view order history	Would like order status notifications
Staff	Inventory and order APIs responded quickly when tested in Swagger UI	Need a proper frontend screen for inventory management (currently API-only)
Admin	Creating and deactivating staff accounts worked smoothly via API	Wants a dashboard with sales analytics and low-stock alerts

10.3 Key takeaways

- Authentication and doctor-listing flows are considered production-ready by all tested users.
- Prescription AI extraction error handling needs a friendlier user-facing message with a retry option.
- Frontend screens for inventory, orders, and admin workflows are the most requested feature for Sprint 2.
- Users were satisfied with the response times of all tested endpoints.

11 Sprint 1 Deliverables Verification

Deliverable	Status	Evidence
OpenAPI 3.1 YAML documentation	Completed	<code>docs/openapi.yaml</code>
API test cases in required format	Completed	Section 9 tables
Input / expected / actual output columns	Completed	Section 9.2 and <code>backend/tests/README.md</code>
Automated pytest submission	Completed	<code>backend/tests/test_*.py</code> (97 tests)
Expected vs actual mismatch demo	Completed	Section 9.3
API Docs page in web app	Completed	<code>/api-docs</code> route (Section 3.2)

12 Next Sprint Plan (Sprint 2)

1. Build frontend screens for inventory management, order processing, and medicine request workflows.
2. Improve user-facing error messages for prescription AI extraction failures with a retry action.
3. Add an API health-check endpoint for environment readiness validation.
4. Implement pagination and filtering on all list endpoints.
5. Add end-to-end tests covering complete customer and admin journeys.
6. Integrate order status change notifications for customers.

13 Conclusion

Sprint 1 delivered a complete, tested, and documented REST API backend for the AI Powered Pharmacy Management System. Ninety-seven automated integration tests pass with zero failures, covering all twelve API domains. An OpenAPI 3.1 YAML file and an in-app Swagger UI page

provide clear API documentation. User testing confirmed that core authentication and doctor-listing flows are production-ready, and the feedback gathered has directly shaped the Sprint 2 backlog.